University of Tabuk

College of Computers & Information Technology

Department of Computer Science

Course Title: CSC 1305 - Theory of Computation

Project Title:

Implementation of Pushdown Automaton (PDA)

Student Name:

Student ID:

Section Number:

Instructor Name:

Submission Date:

Academic Year:

2025 – 2026

Introduction

In the field of computer science, especially in the Theory of Computation, different computational models are used to understand how machines process languages and solve problems. One of the important models is the Pushdown Automaton (PDA). A PDA is an extension of a finite automaton, but it has an additional feature called a stack, which gives it more power compared to DFA and NFA.

The main advantage of the stack is that it allows the automaton to store and retrieve information during the computation process. This makes PDA suitable for solving problems that require memory, such as checking balanced symbols or matching patterns in strings. Because of this ability, PDA is commonly used to recognize context-free languages.

In this project, we focus on designing and implementing a Pushdown Automaton that recognizes a simple but important language, which is $L = \{a^n b^n \mid n \geq 1\}$. This language consists of strings where the number of “a” characters is equal to the number of “b” characters, and all “a” characters come before “b” characters. Examples of valid strings include “ab”, “aabb”, and “aaabbb”, while strings like “abb” or “aab” are not accepted.

This type of language cannot be recognized using a deterministic finite automaton because it requires counting and remembering the number of symbols. However, with the help of the stack in PDA, we can push symbols when reading “a” and pop them when reading “b”, which helps ensure that both counts are equal.

The goal of this project is to design the PDA clearly, explain how it works, and implement it using a programming language. The program will take an input string from the user and determine whether it is accepted or rejected based on the rules of the PDA. This project helps in understanding how theoretical concepts can be applied in practical implementation.

Design

In this project, we designed a Pushdown Automaton (PDA) to recognize the language $L = \{a^n b^n \mid n \geq 1\}$. The main idea of the design is to use a stack to keep track of the number of “a” characters and match them with the same number of “b” characters.

The PDA works in two main phases. In the first phase, the machine reads the input string and pushes a symbol into the stack for every “a” it reads. This means that each “a” is stored in the stack so that it can be counted later. In the second phase, when the machine starts reading “b” characters, it pops one symbol from the stack for each “b”. This ensures that every “b” matches one previously read “a”.

If the number of “b” characters is equal to the number of “a” characters, the stack will be empty at the end of the input. If the stack is empty and the input is fully processed, the string is accepted. Otherwise, the string is rejected.

States of the PDA

The PDA consists of the following states:

- **q0**: Initial state where the machine reads “a” characters and pushes them into the stack.
- **q1**: State where the machine reads “b” characters and pops symbols from the stack.
- **q_accept**: Final state where the string is accepted if all conditions are satisfied.

Transitions

The transitions of the PDA are defined as follows:

- When the machine is in state **q0** and reads “a”, it pushes a symbol (A) into the stack.
- When the machine reads the first “b”, it moves from **q0** to **q1**.
- In state **q1**, for every “b” read, one symbol is popped from the stack.
- If the stack becomes empty exactly when the input ends, the machine moves to the accept state.

- If there are extra symbols in the stack or invalid input order, the string is rejected.

Acceptance Condition

The input string is accepted if:

- All input characters are processed
- The stack is empty at the end
- The input follows the pattern a^nb^n

Otherwise, the string is rejected.

Formal Definition of the PDA

A Pushdown Automaton (PDA) can be formally defined as a 7-tuple:

PDA = (Q, Σ, Γ, δ, q0, Z0, F)

Where each component is explained as follows:

1. Q (Set of States)

Q = {q0, q1, q_accept}

- q0: Initial state (used to read “a” and push into the stack)
- q1: State for reading “b” and popping from the stack
- q_accept: Final state where the input is accepted

2. Σ (Input Alphabet)

Σ = {a, b}

This means the input string consists only of the symbols “a” and “b”.

3. Γ (Stack Alphabet)

Γ = {A, Z0}

- A: Symbol used to represent each “a” stored in the stack
- Z0: Initial symbol at the bottom of the stack

4. δ (Transition Function)

The transition function defines how the PDA moves between states and manipulates the stack.

δ is defined as:

- $\delta(q0, a, Z0) \rightarrow (q0, AZ0)$
- $\delta(q0, a, A) \rightarrow (q0, AA)$
- $\delta(q0, b, A) \rightarrow (q1, \varepsilon)$
- $\delta(q1, b, A) \rightarrow (q1, \varepsilon)$
- $\delta(q1, \varepsilon, Z0) \rightarrow (q\_accept, Z0)$

Explanation:

- When reading “a”, the PDA pushes A onto the stack
- When reading “b”, the PDA pops A from the stack
- ε means no input symbol is read
- When only Z0 remains and input is finished, the string is accepted

5. q0 (Initial State)

q0 is the starting state of the PDA.

6. Z0 (Initial Stack Symbol)

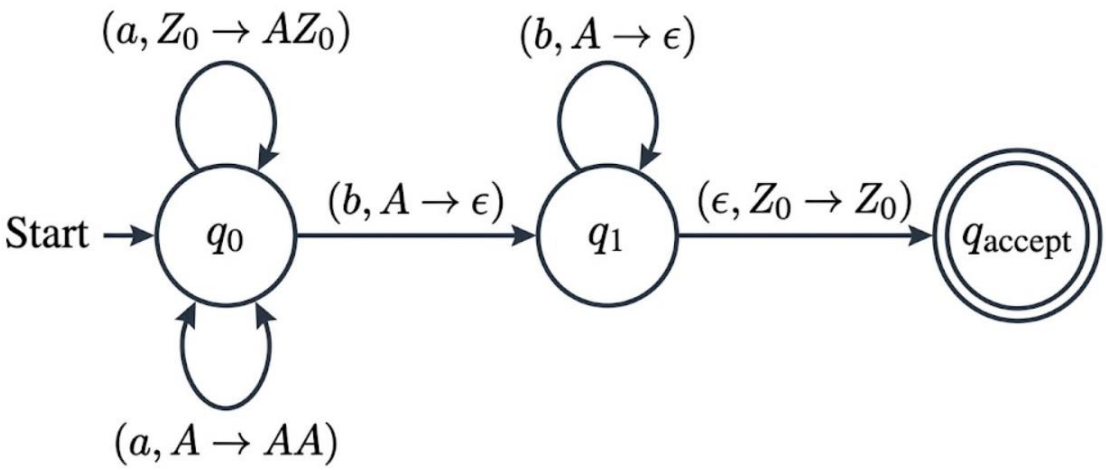
Z0 is the symbol placed at the bottom of the stack at the beginning.

7. F (Set of Final States)

F = {q_accept}

The string is accepted when the PDA reaches this state.

PDA Diagram



PDA Implementation

```
# PDA implementation for language L = {a^n b^n | n >= 1}

def pda_an_bn(input_string):
    stack = []
    state = "q0"

    # push initial stack symbol
    stack.append("Z0")

    for char in input_string:

        # State q0: reading 'a' and pushing into stack
        if state == "q0":
            if char == 'a':
                stack.append('A')

            elif char == 'b':
                # switch to state q1 when 'b' starts
                if stack[-1] == 'A':
                    stack.pop()
```

```
                state = "q1"
            else:
                return "Rejected"

        else:
            return "Rejected"

    # State q1: reading 'b' and popping from stack
    elif state == "q1":
        if char == 'b':
            if stack[-1] == 'A':
                stack.pop()
            else:
                return "Rejected"
        else:
            return "Rejected"

# Final condition check
if state == "q1" and stack == ["Z0"]:
    return "Accepted"
else:
    return "Rejected"


# Main program
print("PDA for L = {a^n b^n | n >= 1}")

user_input = input("Enter a string: ")

result = pda_an_bn(user_input)

print("Result:", result)
```

Code Explanation

The program simulates a Pushdown Automaton using a stack. At the beginning, the stack contains an initial symbol (Z0). When the program reads the input string, it processes each character based on the current state.

In state q0, if the input character is “a”, the program pushes a symbol (A) into the stack. When the first “b” is read, the program changes to state q1 and starts popping symbols from the stack.

In state q1, for every “b”, one symbol is removed from the stack. If at any point the stack does not match the expected condition, the string is rejected.

At the end, if the stack contains only the initial symbol and the input is fully processed, the string is accepted. Otherwise, it is rejected.

Output screenshots

```python
# PDA implementation for language L = {a^n b^n | n >= 1}

Tabnine | Edit | Test | Explain | Document
def pda_an_bn(input_string):
    stack = []
    state = "q0"

    # push initial stack symbol
    stack.append("Z0")

    for char in input_string:

        # State q0: reading 'a' and pushing into stack
        if state == "q0":
            if char == 'a':
                stack.append('A')

            elif char == 'b':
                # switch to state q1 when 'b' starts
                if stack[-1] == 'A':
                    stack.pop()
                    state = "q1"
                else:
                    return "Rejected"

            else:
                return "Rejected"

```

```python
def pda_an_bn(input_string):

        # State q1: reading 'b' and popping from stack
        elif state == "q1":
            if char == 'b':
                if stack[-1] == 'A':
                    stack.pop()
                else:
                    return "Rejected"
            else:
                return "Rejected"

    # Final condition check
    if state == "q1" and stack == ["Z0"]:
        return "Accepted"
    else:
        return "Rejected"


# Main program
print("PDA for L = {a^n b^n | n >= 1}")

user_input = input("Enter a string: ")

result = pda_an_bn(user_input)

print("Result:", result)
```

```python
                if stack[-1] == 'A':
                    stack.pop()
                else:
                    return "Rejected"
            else:
                return "Rejected"

        # Final condition check
        if state == "q1" and stack == ["Z0"]:
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  SPELL CHECKER

PDA for L = {a^n b^n | n >= 1}
Enter a string: ab
Result: Accepted
Result: Accepted

D:\Desktop taha\students assignments 2025\PDA Implementation>C:\Users\ZBOOK\AppData\Local\Programs\Python\Python311\python.exe "d:/Desktop t
PDA for L = {a^n b^n | n >= 1}

D:\Desktop taha\students assignments 2025\PDA Implementation>C:\Users\ZBOOK\AppData\Local\Programs\Python\Python311\python.exe "d:/Desktop t
PDA for L = {a^n b^n | n >= 1}
Enter a string: ba
D:\Desktop taha\students assignments 2025\PDA Implementation>C:\Users\ZBOOK\AppData\Local\Programs\Python\Python311\python.exe "d:/Desktop t
PDA for L = {a^n b^n | n >= 1}
Enter a string: ba
Result: Rejected
Enter a string: ba
Result: Rejected
Result: Rejected
```

Conclusion

In this project, we successfully designed and implemented a Pushdown Automaton (PDA) to recognize the language $L = \{a^n b^n \mid n \geq 1\}$. The main idea was to use a stack to store the number of "a" characters and then match them with the same number of "b" characters.

Through this implementation, we understood how the stack plays an important role in solving problems that require memory. Unlike finite automata, which cannot handle this type of language, the PDA can correctly recognize strings where the number of symbols must be equal.

We also learned how to convert a theoretical concept into a practical program. By writing the code and testing different inputs, we were able to see how the PDA works step by step and how it accepts or rejects strings based on the defined rules.

Overall, this project helped us improve our understanding of automata theory, especially the importance of stack-based computation. It also gave us practical experience in implementing theoretical models using programming, which is very useful for future studies in computer science.